

Data Pre-processing

Understanding What are digital images ?

A **digital image** is simply:

A grid (matrix) of numbers stored inside a NumPy array.

Image can be either A. Greyscale or B. RGB.

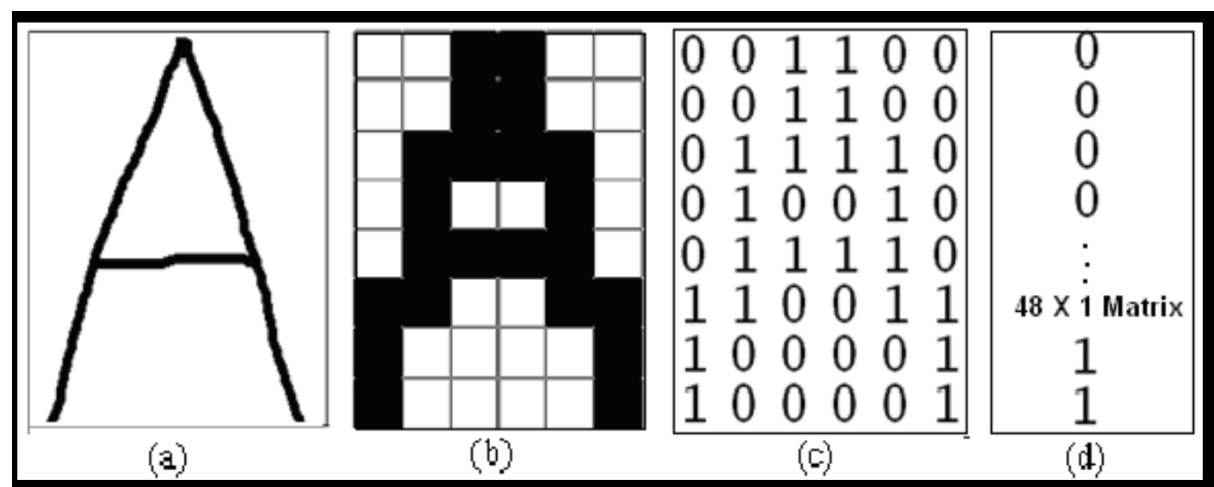
In grayscale:

- Each pixel has **1 value**
- Value range: **0 to 255**
 - 0 → black
 - 255 → white
 - 128 → gray

How do you represent Greyscale image in python ?

```
image = np.array([
    [0, 50, 100],
    [150, 200, 255]
])
```

It has size of (2, 3) Which simple means 2 rows and 3 columns.



RGB means:

- 3 channels:
 - Red
 - Green
 - Blue

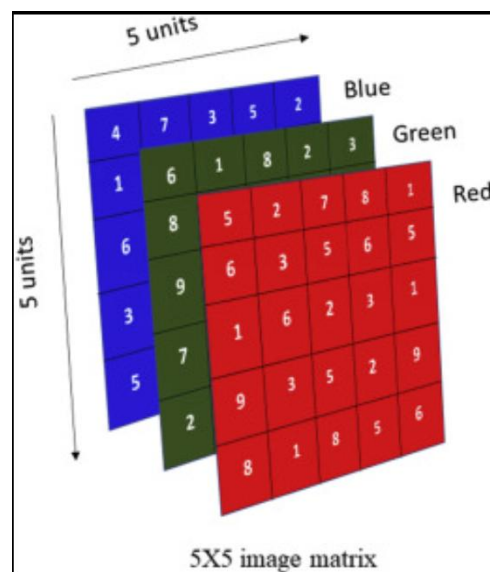
Each pixel has 3 values.

How do you represent RGB image in python ?

```
image = np.array([  
    [[255, 0, 0], [0, 255, 0]],  
    [[0, 0, 255], [255, 255, 0]]  
])
```

It has size of (2, 2, 3). Means, Height = 2, Width = 2, Channels = 3.

So an RGB image is a **3D NumPy array**.



Refer To :

<https://youtu.be/ljc-9L2iXEc?si=VACpBMzKMNYwvo6q>

How do you read image using python ?

1. Using OpenCV

OpenCV is a library.

```
import cv2

image = cv2.imread("leaf.jpg")

print(type(image))
print(image.shape)
```

Output:

```
<class 'numpy.ndarray'>
(224, 224, 3)
```

Important:

OpenCV reads images in **BGR format**, not RGB.

So if you want RGB:

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

2. Using PIL (Python Imaging Library)

Library : PIL (from pillow)

```
from PIL import Image
import numpy as np

image = Image.open("leaf.jpg")

print(type(image)) # PIL Image object

image_array = np.array(image)
print(type(image_array))
print(image_array.shape)
```

PIL reads in **RGB format by default**.

Refer To:

<https://youtu.be/52pMFnkDU-4?si=5wVtIHhxXMKjlyam>

Image Processing using Pillow in Python :

<https://youtu.be/uDNqNv2N-pY?si=xXLLvsC89f-UlrOz>

Image Processing using Scipy in Python: (We won't use this)

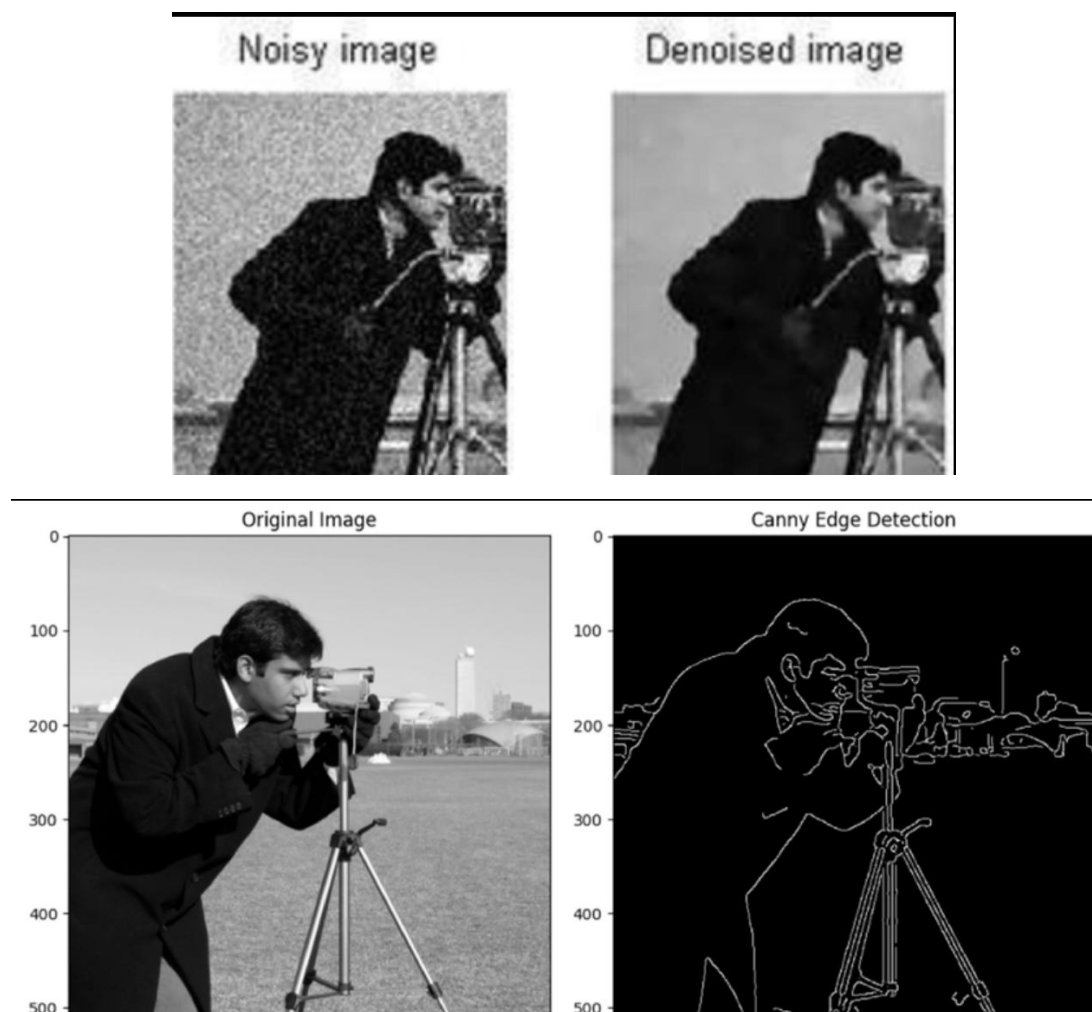
https://youtu.be/s_hDL2fGvow?si=wix1bJdXDvuG5M-J

Reading images, splitting channels, resizing using OpenCV in Python:

<https://youtu.be/yj40XoUqo70?si=Zocf4hJcNbRFjZ>

Denoising and edge detection using OpenCV:

<https://youtu.be/-Qnb8Wv2p1c?si=bbOZagEdxAodtrYh>



How to batch process multiple images in Python ?

Simple steps :

1. Read all image file paths
2. Loop through them
3. Apply preprocessing
4. Save results

Method → Using os

Suppose our folder structure is like this :

```
dataset/  
  image1.jpg  
  image2.jpg  
  image3.jpg
```

And we want to resize each image to 224 x 224:

```
import os  
import cv2  
  
input_folder = "dataset"  
output_folder = "processed"  
os.makedirs(output_folder, exist_ok=True)  
  
for filename in os.listdir(input_folder):  
    if filename.endswith(".jpg"):  
        img_path = os.path.join(input_folder, filename)  
  
        image = cv2.imread(img_path)  
        resized = cv2.resize(image, (224, 224))  
  
        save_path = os.path.join(output_folder, filename)  
        cv2.imwrite(save_path, resized)
```

Now let's see Processing With Subfolders (Important for ML).

Suppose our dataset will be like this :

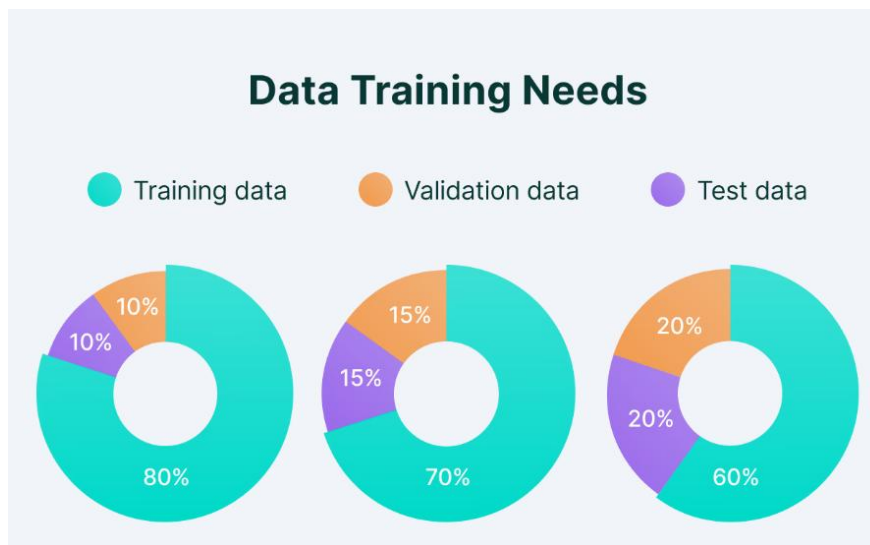
```
dataset/  
    healthy/  
    blight/  
    rust/
```

Then we will follow like this :

```
import os  
import cv2  
  
input_root = "dataset"  
output_root = "processed"  
  
for class_name in os.listdir(input_root):  
    class_path = os.path.join(input_root, class_name)  
  
    if os.path.isdir(class_path):  
        os.makedirs(os.path.join(output_root, class_name), exist_ok=True)  
  
        for filename in os.listdir(class_path):  
            img_path = os.path.join(class_path, filename)  
  
            image = cv2.imread(img_path)  
            resized = cv2.resize(image, (224, 224))  
  
            save_path = os.path.join(output_root, class_name, filename)  
            cv2.imwrite(save_path, resized)
```

Refer To : <https://youtu.be/QxzxLVzNfbl?si=obDJ5zpNxPPEhEoH>

How do we split our data into train, test and validation ?



Refer To : <https://youtu.be/C6wbr1jVVs?si=LypgLD99w6P-EUvg>

Working with large data that doesn't fit your system memory - Semantic Segmentation

Refer To : <https://youtu.be/XhxF2JV6Pcl?si=DKbJJ1u9QtrxgOnw>

Augmentation Techniques

- Rotation
- Horizontal Flip
- Brightness shift
- Zoom
- Crop
- Noise injection

⚠ Do NOT augment validation/test data. (May lead to Data Leakage)

Refer To:

https://youtu.be/kM5jEGsRa4I?si=6_ctlTK-r0hlwVQg

<https://youtu.be/hxLU32zhze0?si=D4GL4ghKLGUz38Hp>